QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

rust-bitcoin @ f127ba2ae14e · rust-bitcoin — public repository (passive review)

Organization	QSI
Report date	2026-11-06 13:50 UTC
Report version	1
Audit run	4ab81962-1c35-43cd-a87a-9e56cda8bf10 · Document audit · 2026-11-06 13:50 UTC
Analyzer engine	qsi-engine 1.3.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`3c66c2f5a80738810e373133f3f6a61ddd2c936670352def1ca38be290728472`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

6	0	0	4	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	0
Long-term post-quantum risk	4
Improvement recommendation	1
Insufficient data	1

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-18748	Medium	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (7 file(s)) preliminary	35.6
QSI-2026-18749	Medium	Quantum-vulnerable ECDSA: transaction and message signing (8 file(s)) preliminary	35.6
QSI-2026-18750	Medium	Quantum-vulnerable ECDSA: role not determined (4 file(s)) preliminary	34.2
QSI-2026-18751	Medium	Quantum-vulnerable Schnorr (BIP-340): role not determined (3 file(s)) preliminary	34.2
QSI-2026-18752	Info	No post-quantum capable cryptographic library detected preliminary	13.3

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/rust-bitcoin/rust-bitcoin @ f127ba2ae14e0447606bfee5d5783e623c423809 — evidence links open the lines at this commit.

Supplied inputs

125 files, 2038425 bytes in total, 18 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.3.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
dependencies	checked	15	Cargo.toml: 0 dependencies parsed (0 pinned); addresses/Cargo.toml: 10 dependencies parsed (0 pinned); base58/Cargo.toml: 3 dependencies parsed (0 pinned)
source_code	checked	110	addresses/src/error.rs: 314 lines scanned with 16 rule patterns and 266 registry patterns; addresses/src/lib.rs: 1608 lines scanned with 16 rule patterns and 266 registry patterns; 291 lines in #[cfg(test)] modules ignored; addresses/src/witness_program.rs: 246 lines scanned with 16 rule patterns and 266 registry patterns; 48 lines in #[cfg(test)] modules ignored
secrets	checked	9	bitcoin/src/address.rs: 1 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name); bitcoin/src/taproot/mod.rs: 2 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name); chacha20_poly1305/src/chacha20.rs: 8 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name)
entropy	checked	1	random-number generation: 110 code files read with 0 macro definitions from the tree; 0 ineffective guard(s), 0 weak random-bytes provider(s), 0 weak key-generation site(s)
osv_advisories	checked	1	39 pinned packages queried at OSV.dev; 1 affected

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 127 (input files: C)

Architecture overview

Static review of the source tree rust-bitcoin at f127ba2ae14e0447606bfee5d5783e623c423809. 125 files analysed; of 250 files, 96 are test, fuzzing, benchmark or example code and were left out, 29 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. The review was restricted to 11 path(s) of the repository: Cargo.toml, addresses, base58, bip158, bitcoin, chacha20_poly1305 and 5 more; 368 files outside them were not read. Files no analyzer reads include: addresses/CHANGELOG.md, addresses/README.md, base58/CHANGELOG.md, base58/README.md, bip158/CHANGELOG.md. No requests were sent to the owner's systems.

Overall security posture

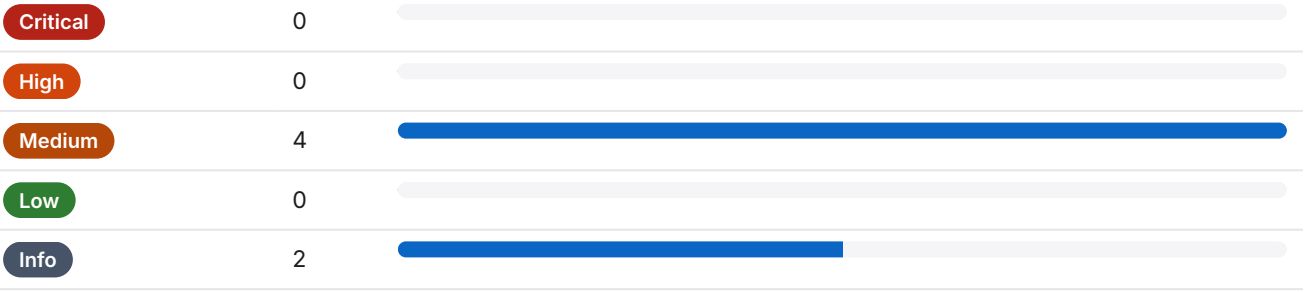
100
Indicative posture index (0–100)

52.1%
Quantum-vulnerable share of inventory

48
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-18748	Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (7 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-18749	Quantum-vulnerable ECDSA: transaction and message signing (8 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-18750	Quantum-vulnerable ECDSA: role not determined (4 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-18751	Quantum-vulnerable Schnorr (BIP-340): role not determined (3 file(s))	Long-term post-quantum risk	Medium	Medium, preliminary
QSI-2026-18752	No post-quantum capable cryptographic library detected	Insufficient data	Info	Medium, preliminary
QSI-2026-18753	bare-metal 0.2.5: informational advisory RUSTSEC-2026-0110 (unmaintained)	Improvement recommendation	Info	Low

Detailed findings

QSI-2026-18748 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): transaction and message signing (7 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

70 reference(s) to Schnorr (BIP-340) across 7 file(s), e.g. addresses/src/lib.rs:123. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:Schnorr (BIP-340):transaction_signing
Data source	addresses/src/lib.rs
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:50 UTC
Priority score	35.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open addresses/src/lib.rs:123 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · addresses/src/lib.rs · line 123 · SHA-256 a62c1a720cf261e2... · File ece00974e73f9eb5...

```
122 |     FullPublicKey, LegacyPublicKey, PubkeyHash, TweakedPublicKey, UntweakedPublicKey,
123 |     XOnlyPublicKey,
124 | };
```

file_excerpt · addresses/src/lib.rs · line 1004 · SHA-256 68f73523d9f7768c... · File ece00974e73f9eb5...

```
1003 |         let payload = self.payload_as_bytes();
1004 |         let xonly_pubkey = XOnlyPublicKey::from(pubkey);
1005 |
```

file_excerpt · addresses/src/lib.rs · line 1015 · SHA-256 876440f5cad2a028... · File ece00974e73f9eb5...

```
1014 |     /// assumed to have already been tweaked.
1015 |     pub fn is_related_to_xonly_pubkey(&self, xonly_pubkey: XOnlyPublicKey) -> bool {
1016 |         xonly_pubkey.serialize().0 == *self.payload_as_bytes()
```

file_excerpt · bitcoin/src/blockdata/script/builder.rs · line 4 · SHA-256 6b19a01bf2ae39d2... · File 07e5a99622d45b1d...

```
  3 | use super::{opcode_to_verify, Error};
  4 | use crate::key::{LegacyPublicKey, XOnlyPublicKey};
  5 | use crate::locktime::absolute;
```

file_excerpt · bitcoin/src/blockdata/script/builder.rs · line 101 · SHA-256 a77d24d3ceefd5bd... · File 07e5a99622d45b1d...

```
100 |         /// Adds instructions to push an XOnly public key onto the stack.
101 |         fn push_x_only_key(self, x_only_key: XOnlyPublicKey) -> Self {
102 |             self.push_slice(x_only_key.serialize().0)
```

file_excerpt · bitcoin/src/crypto/key.rs · line 25 · SHA-256 9b29aaa5edf6e67d... · File 2df23abc0dabfff5...

```
 24 |     SerializedXOnlyPublicKey, TweakedKeypair, TweakedPublicKey, UntweakedKeypair,
 25 |     UntweakedPublicKey, WPubkeyHash, WifKey, XOnlyPublicKey,
 26 | };
```

file_excerpt · bitcoin/src/lib.rs · line 188 · SHA-256 ee9bdfebb3cce7d8... · File f3e202f3d9b18ced...

```
187 |     crypto::key::{
188 |         self, FullPublicKey, Keypair, LegacyPublicKey, PrivateKey, WifKey, XOnlyPublicKey,
189 |     },
```

file_excerpt · bitcoin/src/taproot/mod.rs · line 53 · SHA-256 d552af0f934eb5b2... · File 3f7348eb8976129c...

```
 52 | #[doc(inline)]
 53 | pub use crate::XOnlyPublicKey;
 54 |
```

file_excerpt · bitcoin/src/taproot/mod.rs · line 499 · SHA-256 1df598a45b339923... · File 3f7348eb8976129c...

```
498 |     /// See also [`TaprootBuilder::is_finalizable`]
499 |     pub fn finalize<K: Into<XOnlyPublicKey>>(
500 |         mut self,
```

file_excerpt · bitcoin/src/taproot/mod.rs · line 1109 · SHA-256 388fbad98d219b44... · File 3f7348eb8976129c...

```
1108 |         &self,
1109 |         output_key: XOnlyPublicKey,
1110 |         script: &TapScript,
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18749 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: transaction and message signing (8 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

127 reference(s) to ECDSA across 8 file(s), e.g. bitcoin/src/blockdata/witness.rs:29. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

Affected asset	source-code:ECDSA:transaction_signing
Data source	bitcoin/src/blockdata/witness.rs
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:50 UTC
Priority score	35.6 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open bitcoin/src/blockdata/witness.rs:29 and inspect the quoted lines; the evidence lists further
files.
```

EVIDENCE

file_excerpt · bitcoin/src/blockdata/witness.rs · line 29 · SHA-256 5528dfeaf19f41ae… · File 7516177bb6effefd…

```
   28 |         /// It is expected that `pubkey` is related to the secret key used to create
`signature`.
   29 |         fn p2wpkh(signature: ecdsa::Signature, pubkey: FullPublicKey) -> Self {
   30 |             let mut witness = Witness::new();
```

file_excerpt · bitcoin/src/blockdata/witness.rs · line 70 · SHA-256 1d7c0c34b509a0fd… · File 7516177bb6effefd…

```
   69 |         /// Pushes the DER encoded signature + sighash_type, requires an allocation.
   70 |         fn push_ecdsa_signature(&mut self, signature: ecdsa::Signature) {
   71 |             self.push(signature.serialize())
```

file_excerpt · bitcoin/src/crypto/key.rs · line 12 · SHA-256 c93e86cfaeebca80… · File 2df23abc0dabfff5…

```
   11 | #[rustfmt::skip]                // Keep public re-exports separate.
   12 | pub use secp256k1::{constants, Parity};
   13 | #[cfg(feature = "serde")]
```

file_excerpt · bitcoin/src/crypto/sighash.rs · line 56 · SHA-256 2822beecffec9cda… · File 158eb7428c169de7…

```
   55 |     ($ty:ident) => {
   56 |         impl From<$ty> for secp256k1::Message {
   57 |             fn from(hash: $ty) -> secp256k1::Message {
```

file_excerpt · bitcoin/src/crypto/sighash.rs · line 57 · SHA-256 a44e4b3f206b39e7… · File 158eb7428c169de7…

```
   56 |         impl From<$ty> for secp256k1::Message {
   57 |             fn from(hash: $ty) -> secp256k1::Message {
   58 |                 secp256k1::Message::from_digest(hash.to_byte_array())
```

file_excerpt · bitcoin/src/crypto/sighash.rs · line 58 · SHA-256 25b0f81a19b2e975… · File 158eb7428c169de7…

```
57 |             fn from(hash: $ty) -> secp256k1::Message {
58 |                 secp256k1::Message::from_digest(hash.to_byte_array())
59 |             }
```

file_excerpt · bitcoin/src/lib.rs · line 75 · SHA-256 ab3c659f323740e3... · File f3e202f3d9b18ced...

```
74 | pub extern crate primitives;
75 | pub extern crate secp256k1;
76 | pub extern crate taproot_primitives;
```

file_excerpt · bitcoin/src/sign_message.rs · line 28 · SHA-256 bf4fee8211e6567a... · File 0490974e5abf48fc...

```
27 |     use hashes::sha256d;
28 |     use secp256k1::ecdsa::{RecoverableSignature, RecoveryId};
29 |
```

file_excerpt · bitcoin/src/sign_message.rs · line 63 · SHA-256 e5103bd3d5618003... · File 0490974e5abf48fc...

```
62 |         /// Constructs a new `MessageSignature` from a fixed-length array.
63 |         pub fn from_byte_array(bytes: &[u8; 65]) -> Result<Self, secp256k1::Error> {
64 |             // We just check this here so we can safely subtract further.
```

file_excerpt · bitcoin/src/sign_message.rs · line 66 · SHA-256 daffb40573316105... · File 0490974e5abf48fc...

```
65 |             if bytes[0] < 27 {
66 |                 return Err(secp256k1::Error::InvalidRecoveryId);
67 |             };
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18750 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: role not determined (4 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

16 reference(s) to ECDSA across 4 file(s), e.g. crypto/src/ecdsa.rs:47. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:ECDSA:general
Data source	crypto/src/ecdsa.rs

Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:50 UTC
Priority score	34.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open crypto/src/ecdsa.rs:47 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · crypto/src/ecdsa.rs · line 47 · SHA-256 53c2d800e17a34a8... · File 15b5b705b602cfcc...

```
46 |     /// The underlying ECDSA Signature.
47 |     pub signature: secp256k1::ecdsa::Signature,
48 |     /// The corresponding hash type.
```

file_excerpt · crypto/src/ecdsa.rs · line 55 · SHA-256 e94855bdb686f74e... · File 15b5b705b602cfcc...

```
54 |     #[inline]
55 |     pub fn sighash_all(signature: secp256k1::ecdsa::Signature) -> Self {
56 |         Self { signature, sighash_type: EcdsaSighashType::All }
```

file_excerpt · crypto/src/ecdsa.rs · line 72 · SHA-256 2b73d6e3eb649b2a... · File 15b5b705b602cfcc...

```
71 |         let sighash_type = EcdsaSighashType::from_consensus(u32::from(*sighash_type));
72 |         let signature = secp256k1::ecdsa::Signature::from_der(sig)
73 |             .map_err(|_| InvalidDerError)
```

file_excerpt · crypto/src/lib.rs · line 25 · SHA-256 578776bfe3591a6c... · File 63a929ca613ec617...

```
24 |
25 | pub extern crate secp256k1;
26 |
```

file_excerpt · crypto/src/taproot.rs · line 41 · SHA-256 213d77204e7e7c82... · File 9d693dad36273608...

```
40 |     /// The underlying schnorr signature.
41 |     pub signature: secp256k1::schnorr::Signature,
42 |     /// The corresponding hash type.
```

file_excerpt · crypto/src/taproot.rs · line 57 · SHA-256 0cac400fbdf512ea... · File 9d693dad36273608...

```
56 |             // default type
57 |             let signature = secp256k1::schnorr::Signature::from_byte_array(signature);
58 |             Ok(Self { signature, sighash_type: TapSighashType::Default })
```

file_excerpt · crypto/src/taproot.rs · line 68 · SHA-256 28bd3991c242414a... · File 9d693dad36273608...

```
67 |             }
68 |             let signature = secp256k1::schnorr::Signature::from_byte_array(*signature);
69 |             Ok(Self { signature, sighash_type })
```

file_excerpt · taproot_primitives/src/lib.rs · line 26 · SHA-256 d6300f17aee73f0b... · File 0efe9ae04fe70035...

```
25 | pub extern crate hashes;
26 | pub extern crate secp256k1;
27 |
```

file_excerpt · taproot_primitives/src/lib.rs · line 45 · SHA-256 0a8766074a73453f... · File 0efe9ae04fe70035...

```
44 | use hashes::{hash_newtype, sha256, sha256t, HashEngine as _};
45 | use secp256k1::Scalar;
46 |
```

file_excerpt · taproot_primitives/src/lib.rs · line 428 · SHA-256 0a6bcc9eaac7c6f5... · File 0efe9ae04fe70035...

```
427 |
428 |         let secp_sk = secp256k1::SecretKey::from_secret_bytes(self.to_secret_bytes())
429 |             .expect("secret key parsed from serialized secret key");
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18751 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Schnorr (BIP-340): role not determined (3 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

8 reference(s) to Schnorr (BIP-340) across 3 file(s), e.g. crypto/src/lib.rs:33. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:Schnorr (BIP-340):general
Data source	crypto/src/lib.rs
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:50 UTC
Priority score	34.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open crypto/src/lib.rs:33 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · crypto/src/lib.rs · line 33 · SHA-256 9415f48bf1c10122… · File 63a929ca613ec617…

```
32 | #[doc(inline)]
33 | pub use self::key::{FullPublicKey, Keypair, LegacyPublicKey, PrivateKey, XOnlyPublicKey};
34 |
```

file_excerpt · crypto/src/taproot.rs · line 41 · SHA-256 213d77204e7e7c82… · File 9d693dad36273608…

```
40 |     /// The underlying schnorr signature.
41 |     pub signature: secp256k1::schnorr::Signature,
42 |     /// The corresponding hash type.
```

file_excerpt · crypto/src/taproot.rs · line 57 · SHA-256 0cac400fbdf512ea… · File 9d693dad36273608…

```
56 |             // default type
57 |             let signature = secp256k1::schnorr::Signature::from_byte_array(signature);
58 |             Ok(Self { signature, sighash_type: TapSighashType::Default })
```

file_excerpt · crypto/src/taproot.rs · line 68 · SHA-256 28bd3991c242414a… · File 9d693dad36273608…

```
67 |             }
68 |             let signature = secp256k1::schnorr::Signature::from_byte_array(*signature);
69 |             Ok(Self { signature, sighash_type })
```

file_excerpt · taproot_primitives/src/lib.rs · line 42 · SHA-256 7bbdbc3b72b9163e… · File 0efe9ae04fe70035…

```
41 | use crypto::key::{
42 |     TweakedKeypair, TweakedPublicKey, UntweakedKeypair, UntweakedPublicKey, XOnlyPublicKey,
43 | };
```

file_excerpt · taproot_primitives/src/lib.rs · line 425 · SHA-256 73e61752c16ba724… · File 0efe9ae04fe70035…

```
424 |     fn tap_tweak(&self, merkle_root: Option<TapNodeHash>) -> TweakedKeypair {
425 |         let pubkey = XOnlyPublicKey::from_keypair(self);
426 |         let tweak = TapTweakHash::from_key_and_merkle_root(pubkey, merkle_root).to_scalar();
```

file_excerpt · taproot_primitives/src/lib.rs · line 431 · SHA-256 98da70df00a22c65… · File 0efe9ae04fe70035…

```
430 |         let (xonly, parity) = self.to_x_only_public_key().serialize();
431 |         let secp_pk = secp256k1::XOnlyPublicKey::from_byte_array(xonly)
432 |             .expect("x-only public key parsed from serialized point");
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to a post-quantum algorithm; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18752 · DEP-NO-PQC-LIBRARY

Info

No post-quantum capable cryptographic library detected

Insufficient data | Confidence: Medium | preliminary — requires further verification

Across 15 dependency manifests, QSI found crypto libraries (secp256k1) but none from its PQC-capable list.

Affected asset	dependencies
Data source	dependency manifests
Detection method	Dependency name matching.
Analyzer	dependencies 1.3.0 · 2026-11-06 13:50 UTC
Priority score	13.3 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

Readiness indicator only: PQC support may come from the platform (OpenSSL, OS, cloud KMS) rather than application dependencies.

REPRODUCTION STEPS

```
Review the dependency manifests listed in the evidence.
```

EVIDENCE

dependency · Cargo.toml, addresses/Cargo.toml, base58/Cargo.toml, bip158/Cargo.toml, bitcoin/Cargo.toml, bitcoin/embedded/Cargo.lock, bitcoin/embedded/Cargo.toml, chacha20_poly1305/Cargo.toml, crypto/Cargo.toml, hashes/Cargo.toml, hashes/embedded/Cargo.lock, hashes/embedded/Cargo.toml, key_expression/Cargo.toml, primitives/Cargo.toml, taproot_primitives/Cargo.toml · SHA-256 287cb2008737c0fe…

```
Crypto libraries: secp256k1
```

LIMITATIONS OF THE CHECK

Absence of evidence is not evidence of absence: system libraries and transitive dependencies may not be listed.

RECOMMENDATION

Confirm where PQC support will come from (TLS terminator, KMS/HSM, library upgrade).

RETEST PROCEDURE

Re-scan after upgrades.

FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM); FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA)

QSI-2026-18753 · DEP-ADVISORY-INFORMATIONAL

Info

bare-metal 0.2.5: informational advisory RUSTSEC-2026-0110 (unmaintained)

Improvement recommendation | Confidence: Low

OSV.dev lists RUSTSEC-2026-0110 for crates.io package bare-metal at version 0.2.5 as informational (unmaintained): bare-metal is deprecated. It reports no vulnerability; an unmaintained package will not receive fixes for future ones.

Affected asset	dependency:crates.io/bare-metal@0.2.5
Data source	OSV.dev
Detection method	OSV.dev querybatch API lookup of the exact declared version.
Analyzer	dependencies 1.3.0 · 2026-11-06 13:50 UTC
Priority score	11.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable

POTENTIAL IMPACT

No known vulnerability. Future security fixes may not be released.

REPRODUCTION STEPS

```
curl -s https://api.osv.dev/v1/vulns/RUSTSEC-2026-0110
```

EVIDENCE

tool_output · OSV.dev RUSTSEC-2026-0110 · manifest · SHA-256 1377cbc63847011c...


```
{"id": "RUSTSEC-2026-0110", "aliases": [], "modified": "2026-04-24T09:45:08.136572Z", "summary":
"bare-metal is deprecated", "severity": null, "declared_in_manifest": false}
```


LIMITATIONS OF THE CHECK

Advisory data comes from OSV.dev at the recorded time.

RECOMMENDATION

Plan a move to a maintained alternative, or record the decision to keep it.

RETEST PROCEDURE

Re-run the dependency scan after the change.

OSV advisory RUSTSEC-2026-0110

Evidence

ID	Type	Data source	SHA-256
QSI-2026-18748	file_excerpt	addresses/src/lib.rs (line 123)	a62c1a720cf261e21ac1a7f790debb52b0916021246b022bc74e44169045b4
QSI-2026-18748	file_excerpt	addresses/src/lib.rs (line 1004)	68f73523d9f7768c7fca900284aaf5ad49588cb1e7f457a1e4716378479f9e
QSI-2026-18748	file_excerpt	addresses/src/lib.rs (line 1015)	876440f5cad2a028e70c4ebfe7a651a4359c53478ca7fca746bb8f9fbe7330
QSI-2026-18748	file_excerpt	bitcoin/src/blockdata/script/builder.rs (line 4)	6b19a01bf2ae39d2c14347241bfdf5eed393a37b231cbd0b916f78b9594071
QSI-2026-18748	file_excerpt	bitcoin/src/blockdata/script/builder.rs (line 101)	a77d24d3ceefd5bd00ea7f4f819009ee0569035d33b14dc022cc01dfd9f60b
QSI-2026-18748	file_excerpt	bitcoin/src/crypto/key.rs (line 25)	9b29aaa5edf6e67d448fc20542ef14d9ffe43528a53cb568a9223fb0a9c6d6
QSI-2026-18748	file_excerpt	bitcoin/src/lib.rs (line 188)	ee9bdfebb3cce7d866f2387bc9bf123f38b960224e8243d4277a1bf598ae22
QSI-2026-18748	file_excerpt	bitcoin/src/taproot/mod.rs (line 53)	d552af0f934eb5b2183e1192b5d537bed29099ae7e023db4ba5506ee91614b
QSI-2026-18748	file_excerpt	bitcoin/src/taproot/mod.rs (line 499)	1df598a45b339923eeba62cb70baac0769ee322b2315d2ea2a24bd8093b46b
QSI-2026-18748	file_excerpt	bitcoin/src/taproot/mod.rs (line 1109)	388fbad98d219b448d192aaa35351052f357b8b9ada089a9cfee234b1921e0
QSI-2026-18749	file_excerpt	bitcoin/src/blockdata/witness.rs (line 29)	5528dfeaf19f41ae8b442a836d01fb1af70f710b67c36685d78ec42ae43ee6
QSI-2026-18749	file_excerpt	bitcoin/src/blockdata/witness.rs (line 70)	1d7c0c34b509a0fd4a4fbb9afb78e3a30a70d8c71254c3249c64bfb849505d
QSI-2026-18749	file_excerpt	bitcoin/src/crypto/key.rs (line 12)	c93e86cfaeebca80fbf060ce376c25680c18aa75440b4253ffcd7a15a541f9
QSI-2026-18749	file_excerpt	bitcoin/src/crypto/sighash.rs (line 56)	2822beecffec9cdaf16cd469a9b483c5f199eabb1a5112d154fb772fefc005
QSI-2026-18749	file_excerpt	bitcoin/src/crypto/sighash.rs (line 57)	a44e4b3f206b39e7373c5f48d480336f2629628dd7187b405314a5cc90da0c
QSI-2026-18749	file_excerpt	bitcoin/src/crypto/sighash.rs (line 58)	25b0f81a19b2e9754b744b8d912d6f6de45858b0c60259a7cd27242a84ba2c
QSI-2026-18749	file_excerpt	bitcoin/src/lib.rs (line 75)	ab3c659f323740e32e371c569c0f6ac7f7ff86e25ad6ec754da5d97e126b58

QSI-2026-18749	file_excerpt	bitcoin/src/sign_message.rs (line 28)	bf4fee8211e6567a4e0a3a3739c0f3c72177fc774e7f45999f64f4cdf35bd4
QSI-2026-18749	file_excerpt	bitcoin/src/sign_message.rs (line 63)	e5103bd3d5618003bfda34ed283bcfa72b5be925a6f77e78e667ea6b0d7acd
QSI-2026-18749	file_excerpt	bitcoin/src/sign_message.rs (line 66)	daffb40573316105a3cbc121be9b542304e26171369749552d75f17aa3ad69
QSI-2026-18750	file_excerpt	crypto/src/ecdsa.rs (line 47)	53c2d800e17a34a85eb1592b1e52421478bf84dc95d463f9736abce9ab62b6
QSI-2026-18750	file_excerpt	crypto/src/ecdsa.rs (line 55)	e94855bdb686f74e258a508f15131f1590dbac165bd7028d1cf0730771b56a
QSI-2026-18750	file_excerpt	crypto/src/ecdsa.rs (line 72)	2b73d6e3eb649b2a23602e482ce344e339f148dd5f49125fea88808d2268ee
QSI-2026-18750	file_excerpt	crypto/src/lib.rs (line 25)	578776bfe3591a6c57fcfeb143276ee3d3b8df7a8a565dc7d054a3f0b1f368
QSI-2026-18750	file_excerpt	crypto/src/taproot.rs (line 41)	213d77204e7e7c822521a037fd89f4e9af3fd172cf63282da563512204b11d
QSI-2026-18750	file_excerpt	crypto/src/taproot.rs (line 57)	0cac400fbdf512ea35e6d74060c50a98fc42199dd184dee36f1f92376f161c
QSI-2026-18750	file_excerpt	crypto/src/taproot.rs (line 68)	28bd3991c242414afc776fa4fc386bf16462ff632f79400412ceebcf8f0a8f
QSI-2026-18750	file_excerpt	taproot_primitives/src/lib.rs (line 26)	d6300f17aee73f0bd828c9fdcb6ce3ddae6425ddcdfa65f65c3681722964a7
QSI-2026-18750	file_excerpt	taproot_primitives/src/lib.rs (line 45)	0a8766074a73453f2b6314ff9afd59a6df105eda4b2c7153ec595c3107e689
QSI-2026-18750	file_excerpt	taproot_primitives/src/lib.rs (line 428)	0a6bcc9eaac7c6f524ec18cb59f4f7a67f61f2bcb70890f81c37a61fddefe5
QSI-2026-18751	file_excerpt	crypto/src/lib.rs (line 33)	9415f48bf1c101229771f715c5938d47e0fd714ba4311f88b6fede66e0b5cc
QSI-2026-18751	file_excerpt	crypto/src/taproot.rs (line 41)	213d77204e7e7c822521a037fd89f4e9af3fd172cf63282da563512204b11d
QSI-2026-18751	file_excerpt	crypto/src/taproot.rs (line 57)	0cac400fbdf512ea35e6d74060c50a98fc42199dd184dee36f1f92376f161c
QSI-2026-18751	file_excerpt	crypto/src/taproot.rs (line 68)	28bd3991c242414afc776fa4fc386bf16462ff632f79400412ceebcf8f0a8f
QSI-2026-18751	file_excerpt	taproot_primitives/src/lib.rs (line 42)	7bbdbc3b72b9163efc341a1a788fdea69b6bfd512150379b197fe7ac71a9d9
QSI-2026-18751	file_excerpt	taproot_primitives/src/lib.rs (line 425)	73e61752c16ba7243668d77331629a9c2d91098e96035447a32b3717ed3c21

QSI-2026-18751	file_excerpt	taproot_primitives/ src/lib.rs (line 431)	98da70df00a22c65acf45d929606a3e768b6ec321491229888a44afd4b7971
QSI-2026-18752	dependency	Cargo.toml, addresses/ Cargo.toml, base58/ Cargo.toml, bip158/ Cargo.toml, bitcoin/ Cargo.toml, bitcoin/ embedded/ Cargo.lock, bitcoin/ embedded/ Cargo.toml, chacha20_poly1305/ Cargo.toml, crypto/ Cargo.toml, hashes/ Cargo.toml, hashes/ embedded/ Cargo.lock, hashes/ embedded/ Cargo.toml, key_expression/ Cargo.toml, primitives/ Cargo.toml, taproot_primitives/ Cargo.toml	287cb2008737c0fe03b7729193c577d77a128b5d87065e91d8e4624b7d0f14
QSI-2026-18753	tool_output	OSV.dev RUSTSEC-2026-0110 (manifest)	1377cbc63847011c54324d486e3a7bae12ccbda6c1c9a981451db3532783f3

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
transaction and message signing	ECDSA (8), Schnorr (BIP-340) (7)	QSI-2026-18748, QSI-2026-18749	Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them
role not determined	ECDSA (4), Schnorr (BIP-340) (3)	QSI-2026-18750, QSI-2026-18751	Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
weak classically	2

reduced margin (Grover), approved	11
Grover: acceptable to NIST, below ECCG/ANSSI guidance	5
n/a	5
vulnerable (Shor)	25

Detailed findings: QSI-2026-18748, QSI-2026-18749, QSI-2026-18750, QSI-2026-18751

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDSA (15), Schnorr (BIP-340) (10)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.
Grover's algorithm — key search and preimages Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	ChaCha20-Poly1305 (2), HKDF (1), HMAC (6), Poly1305 (1), RIPEMD-160 (1), SHA-1 (1), SHA-2 (6)	Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance.
Brassard–Høyer–Tapp — collision search Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	RIPEMD-160 (1), SHA-1 (1), SHA-2 (6)	256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Remediation plan

QSI-2026-18748 · P3 · 90 days

short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18749 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18750 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18751 · P3 · 90 days

short-term risk reduction

Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18748 · P4 · 90 days

root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-18749 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	

	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-18750 · P4 · 90 days

root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-18751 · P4 · 90 days

root-cause fix

Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-18753 · P4 · 90 days

root-cause fix

Replace bare-metal with a maintained alternative or record why it stays.

Rationale	Eliminates the root cause described in the finding.
Expected outcome	The weakness is no longer present on re-test.
Complexity	medium
Dependencies	Change window and owner approval.
Side effects	Code changes may be required.
Tests before change	Reproduce the finding using the documented steps.
Tests after change	Re-run the retest procedure from the finding.
Closure criteria	bare-metal replaced, or the decision recorded.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
None			

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-18748	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18749	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18750	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18751	Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-18748	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18749	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18750	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18751	Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18753	Replace bare-metal with a maintained alternative or record why it stays.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-18748	Re-run the inventory after refactoring.
QSI-2026-18749	Re-run the inventory after refactoring.
QSI-2026-18750	Re-run the inventory after refactoring.
QSI-2026-18751	Re-run the inventory after refactoring.
QSI-2026-18752	Re-scan after upgrades.
QSI-2026-18753	Re-run the dependency scan after the change.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.2.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.3.0
inventory_document	1.1.0
solidity	1.3.0
source_code	1.4.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.3.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
2. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
3. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
4. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
5. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
6. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
7. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
8. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
9. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
10. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
11. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
12. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
13. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
14. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
15. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
16. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
17. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/final (final (2026-10-27; the initial public draft of November 2024 is obsoleted). Tables 2 and 4: ECDSA, EdDSA and RSA signatures, finite-field and elliptic-curve DH and MQV, and RSA key establishment are deprecated after 2030 and disallowed after 2035 at every key size — the draft deprecated only the 112-bit level. A composite (hybrid) key establishment stays FIPS 140-validatable while one input comes from an approved scheme. Repeats the federal deadlines of OMB M-26-15: post-quantum key establishment by 2030-12-31 and signatures by 2031-12-31 for high value assets and high impact systems)
18. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
19. OSV advisory RUSTSEC-2026-0110 — OSV.dev. https://osv.dev/vulnerability/RUSTSEC-2026-0110 (modified 2026-04-24T09:45:08.136572Z)
20. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)

21. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)
22. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Quantum status	Classical status	Count	Locations	Examples
RIPEMD-160	RIPEMD-160	weak classically	legacy	1	1	hashes/src/lib.rs:126
SHA-1	SHA-1	weak classically	deprecated	1	1	hashes/src/sha1/crypto.rs:32
ChaCha20-Poly1305		reduced margin (Grover), approved	acceptable	1	1	chacha20_poly1305/src/chacha20.rs
ChaCha20-Poly1305	ChaCha20-Poly1305	reduced margin (Grover), approved	acceptable	1	1	chacha20_poly1305/src/lib.rs:37
HKDF		reduced margin (Grover), approved	acceptable	1	1	hashes/src/lib.rs:115
HMAC		reduced margin (Grover), approved	acceptable	4	4	hashes/src/hkdf/mod.rs:13; hashes/src/hmac/mod.rs:85; hashes/src/lib.rs:116
HMAC	HMAC-SHA-256	reduced margin (Grover), approved	acceptable	1	1	hashes/src/lib.rs:156
HMAC	HMAC-SHA-512	reduced margin (Grover), approved	acceptable	1	1	hashes/src/lib.rs:159
Poly1305		reduced margin (Grover), approved	acceptable	1	1	chacha20_poly1305/src/poly1305.rs
SHA-2	SHA-512	reduced margin (Grover), approved	acceptable	1	1	hashes/src/sha512/crypto.rs:100
SHA-2	SHA-256	Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable	5	5	hashes/src/sha256/crypto/arm_sha2.rs:29; hashes/src/sha256/crypto/avx2.rs:164; hashes/src/sha256/crypto/mod.rs:215
Crypto library secp256k1 (secp256k1 bindings)	0.33.0	n/a	acceptable	5	5	bitcoin/Cargo.toml; bitcoin/embedded/Cargo.lock; crypto/Cargo.toml

ECDSA		vulnerable (Shor)	acceptable	4	4	bitcoin/src/blockdata/witness.rs:29; bitcoin/src/crypto/sighash.rs:91; crypto/src/ecdsa.rs:47
ECDSA	secp256k1	vulnerable (Shor)	acceptable	11	11	bitcoin/src/crypto/key.rs:12; bitcoin/src/crypto/sighash.rs:56; bitcoin/src/lib.rs:75
Schnorr (BIP-340)		vulnerable (Shor)	acceptable	10	10	addresses/src/lib.rs:123; bitcoin/src/blockdata/script/builder.rs:4; bitcoin/src/crypto/key.rs:25

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

B. JSON

qsi.report/1 · SHA-256 3c66c2f5a80738810e373133f3f6a61ddd2c936670352def1ca38be290728472

C. Input files

File	Size	SHA-256	Redactions
Cargo.toml	5373	e8bf29d651f5c6ee4c292a3402614f8ece9c767dedcdd4a2ed6b0d3eb88d7cf5	0
addresses/Cargo.toml	2013	3bd9f423606d7458fd2c255dc59307d5dc61ec7626888299c28f518f8b3aa697	0
addresses/src/error.rs	9710	ad67a11a320f725c1dbb103da0b1f1e7ee3d633bbc9f4f293f9f0feabc7c4073	0
addresses/src/lib.rs	66824	ece00974e73f9eb54b931fcbf26fa63fa348dbd560546cfa2871b2a7843e161a	0
addresses/src/witness_program.rs	9228	b8e1d242636be622dad37f6d2f272cf4a822c5f96d93ac162d85e01b9ed8e36b	0
base58/Cargo.toml	1012	d837e0d7392179c2fc996700cfdfb8d9e8cd6f4d48aa6b3304b3002da3b12dae	0
base58/src/error.rs	11149	8a76188cc5f1a0eb9102502f0810740f0525a03d32fc66339daf5e43c6794e67	0
base58/src/ext_array_vec.rs	3330	355f940b6980a81af463032e9d6aecf9302afbd2620d8c2ba661ed8c9d3c81d7	0
base58/src/lib.rs	25702	353388dadd79e5167b35c162b29f675a7dd06ec83219176331565ee2fad7089e	0
bip158/Cargo.toml	1469	eef49bdb47c4934084695b3a06fa29c3efea18157076e8057a88c20d28277e88	0
bip158/src/filter.rs	21845	533e291cc898fdb4dd255e34546ba30061e31d760761a21244c69db249d0939e	0
bip158/src/hash_types.rs	4682	76677bcf066aa24651c3df1816adde3c74cd59d0439db21d9910a16f598252c5	0
bip158/src/lib.rs	2709	db46c5a6966e9ead82c845b73aba2f13063b848b449ba453626844c46e8d496b	0
bitcoin/Cargo.toml	5122	fb516481d3eaf37d66f8eb5abace88706504f4ec910d6514578169a704a32425	0
bitcoin/embedded/Cargo.lock	11126	f5b10832abfbfd8f6f1f216747d473de2ce9af006a2d936704da806a38588653	0
bitcoin/embedded/Cargo.toml	897	955fd6ff50f61fe797ebd2ae6bc5efa4730df4c06126b007096b69462963b20c	0

bitcoin/embedded/src/main.rs	1484	29075044edc0e8c72c5ad9358290cd41a1605ce414801230988da4784bc920c6	0
bitcoin/src/address.rs	16897	66b5446df5096dbe6074784b497c0f9441f510171d8283ffc7472330f7b42cc1	1
bitcoin/src/bip158.rs	23408	817e7965f048d514fbb746cb670c65114554e026feaf7195a1a5eb2fd2c5660f	0
bitcoin/src/blockdata/block.rs	29237	f42a9c15f772b4adc72ad67b8b6eabd69a1224c6f415b8a6fcbc24a139868c40	0
bitcoin/src/blockdata/constants.rs	18990	fdf80b8a5eb0ae04fc82a101d7e63d52f460ca44084a1b4fec92e8939407d593	0
bitcoin/src/blockdata/mod.rs	3063	f75808cb3a5704725756c7d9f627126ef2681ce2437bdef860f81358dff8c366	0
bitcoin/src/blockdata/opcodes.rs	41605	f07eafaf98fec6cedaffd7a8dea2b12ee45cced5be59059561010ef05f413eae	0
bitcoin/src/blockdata/script/borrowed.rs	23945	c6645b184ed13c98bf834a4319890f0f6f74287f9c3d09f97f3f1750259ff0cc	0
bitcoin/src/blockdata/script/builder.rs	6878	07e5a99622d45b1d598c69b68b20f061cce279e95156c742cb1852b57f0217d9	0
bitcoin/src/blockdata/script/instruction.rs	9661	6910dc6b75e179e849878c31d1281837965171e0efe4aba3030209040c5e0e32	0
bitcoin/src/blockdata/script/mod.rs	10542	d7c50a4889de1ecd5d3115338fb32d8b8ca736ff6f93f66d4341fc969c8b2add	0
bitcoin/src/blockdata/script/owned.rs	13066	6ec49656f2f33d57d406547612f93cce95a50c91e7e58bf7efe22346754efb1a	0
bitcoin/src/blockdata/script/push_bytes.rs	4959	4e88c38df3ecffb89d8411f5c38b92dfb12131da86270985d6266434b848c4f3	0
bitcoin/src/blockdata/script/script_code.rs	1709	bef241d9842a104af8e2a14258548218fe4b4dbd9d00b82d8e753b65499e46ac	0
bitcoin/src/blockdata/script/witness_version.rs	3013	6f45000dd8ec46d35414a37851dbff91a30b9a00d74ecf3d6b15d981ef74cb92	0
bitcoin/src/blockdata/transaction.rs	88053	a78e7f37d17c90fc07e2cf7f597bf7cb91c0b71f26433bbecc717be9e904901d	0
bitcoin/src/blockdata/witness.rs	20870	7516177bb6effefd27a72bb846592688635569 82c8766d2455d8b54f39fa71f4	0
bitcoin/src/consensus_validation.rs	9904	c4c2ed94e5ad3b4791094b5393352365b6b555d88e72262552271c69f6c03103	0
bitcoin/src/crypto/key.rs	4794	2df23abc0dabfff5cbcb1676eb835acc4c1b63ac4bd5c70b626c0f4a6118bfd3	0
bitcoin/src/crypto/mod.rs	591	d7ede9e715b6de219217605542a6463c5f6c578b22fff8599b19da391fb25ed7	0
bitcoin/src/crypto/sighash.rs	92044	158eb7428c169de7f0617db2c82357e2a53afb669e2e06ceae5f008c3b9b168d	0
bitcoin/src/hash_types.rs	3522	183a8cd9c6ce6db95f21f4d35bcc75d877a8b39d62988db191a5ae7e0098a83e	0

bitcoin/src/internal_macros.rs	2993	60544e3927dfb93b1341adb74356758a10d4f5c38a9fad5eaff01e495b91a891	0
bitcoin/src/lib.rs	11363	f3e202f3d9b18ced89e127f6868f1abf5d83c2011071ed894f989e9252520049	0
bitcoin/src/merkle_tree/mod.rs	957	ada941367ce3c12fd025317af125e479348e83a59824b9dd00fb751a2e63aca9	0
bitcoin/src/network/mod.rs	3952	e61971081da1dd960c196e5bed715f1da47d9a75d48887fa8a37e76f1a5ad049	0
bitcoin/src/network/params.rs	12479	3f116d15831a48fc87cc5b697436b647881aa3321aa24fc82fba434f713ef9bd	0
bitcoin/src/policy.rs	2601	bc7ef91de58e80e0e2108e24f22fc370cdd8e7567c2df7894df0e4d2df37f9eb	0
bitcoin/src/pow.rs	36399	533f346853985f4ad5927e4ee099e499af04a7b11260df45176f228858914261	0
bitcoin/src/sign_message.rs	14034	0490974e5abf48fc440341969267e3054cf9d211ea053a04796c84318ea8afa3	0
bitcoin/src/taproot/merkle_branch/borrowed.rs	10201	0f05c94d9865395c46f806cc3cf44121d8938dceb86a607f6dfe78bc95c26119	0
bitcoin/src/taproot/merkle_branch/buf.rs	9107	57e6dbc78d2a1d9c385073aa3d19790303cf519610b93fa83001a76e156a6ff7	0
bitcoin/src/taproot/merkle_branch/mod.rs	2638	7e2a80098101843975f596a5e547257dbe199180b41cc3745a68e110c032a124	0
bitcoin/src/taproot/mod.rs	84505	3f7348eb8976129c7bca923770d3738026d7892d021593ed394364cefa7540ea	2
chacha20_poly1305/Cargo.toml	774	9c7b67669ae343d2f19ced0fc8d9c9472604332ca51bb143cb780f3bc48da854	0
chacha20_poly1305/src/chacha20.rs	30677	c21cbe5b9fe807284d2cd3e65787725e439f36412ad7a0aee96f66e9595527e3	8
chacha20_poly1305/src/chacha20/avx2.rs	13746	be640d2626aa4ee2466b626cc78129a150ab39f800dc1932d51e1b23de7339a7	1
chacha20_poly1305/src/chacha20/neon.rs	19982	2c1dfdf324d4e110955ae984b39dc6d86514479e646f2f3922085c2c38d92b91	2
chacha20_poly1305/src/chacha20/sse2.rs	12404	43b1abcbcb85df34a9d6b5996fbc7b0dc24375c5579b3c311d05606fd16d77ba	1
chacha20_poly1305/src/lib.rs	8259	285c8255e6dde2bbb615b0257e12d449db4b340c7a88bf1726c1390642e984c0	1
chacha20_poly1305/src/poly1305.rs	9985	acf7d6f169d5277c7799915ae6ed1d90b3389edabeae8b6cfbb00012266f7c0f	1
crypto/Cargo.toml	2128	0b2764683d818c132a6e54a72f770e52488650c2d2181ceaf6dc508d50f0b85d	0
crypto/src/ecdsa.rs	16680	15b5b705b602cfccdc101dc668c9f1e1dac3a98e9b0e5b02417b3b306f59b79b	0
crypto/src/key.rs	97439	e6ec0c249f979301ec4102cd05e208bfd3697f48de38c7ebea85808d17721713	1
crypto/src/lib.rs	1936	63a929ca613ec617c03b443f48baa2eddfd00c66967ed0eee58ff13c3ac08510	0
crypto/src/sighash.rs	19617	631667280ad0756069ca6eac0a3874dd2805a241d84a199ffd29be748cb142e6	0

crypto/src/taproot.rs	21754	9d693dad362736086d00a7d8f8d36592f7629ba6560ffcd550f680929e6f9150	0
hashes/Cargo.toml	1823	3a81da5cf718beec623aa609e35511acc888b8d30cb893ec0cc26d0fbac62981	0
hashes/embedded/Cargo.lock	7256	23e5c2173fbc8554c909edfd4e64a761591a562d4dd41204a6b4b324ccdc9de9	0
hashes/embedded/Cargo.toml	1089	ec0da86761d2af6227cbed506de3928556db677078ed3bb444dfcec346a0d7ab	0
hashes/embedded/src/main.rs	1926	465f12a2e8ba2641f8c18543eaa885a0dac2a4fa1101e6a4a40a641af0579144	0
hashes/src/cmp.rs	4405	5e2856045a9b18bf176b3aae12336b23fa1aba73ed36e3e61aaf3fdeda34af46	0
hashes/src/hash160/mod.rs	4414	0b141e2362abcd7bfad058a8f834c2a67bd8c800e2fcd03c788192c53d013f20	0
hashes/src/hkdf/mod.rs	10569	bef046b7d422595aea0727a1922b9439574c8787822d4d27ab85dffe872fb019	0
hashes/src/hmac/mod.rs	12860	f9d6d941441682bb8d915c6d94fb1a0d0702256f9a884111ec6e09b19dd6b72d	0
hashes/src/internal_macros.rs	8424	b694e44d79ac571e25ded16cb3f0c9f6a0f87883c3e2aacea06a147ceebac3a8	0
hashes/src/lib.rs	10878	8330d3eeac3ff37774830e8a51512dc1fe57cc9f6a433ed2e5dfaa1eedecc36e	0
hashes/src/macros.rs	26846	7225b9d7f80cdf831807107e4f44afcdc0e1613ac46706ec5d8a174d67dea9a3	0
hashes/src/muhash/mod.rs	1523	795e2a9dcf7ec438125eeea2ed2dd7f6a18a42dd2eade559499f66a839833979	0
hashes/src/ripemd160/crypto.rs	19744	7e593401bf1d5ea7dd3d80f020a493366e9cdbaa1cbc5d14f89bbfcbdbe3b51e	0
hashes/src/ripemd160/mod.rs	2501	e112a478fc820538876841cb112f8b8a0be1d1eb7a6aab0f51ce4637ae4602a9	0
hashes/src/sha1/crypto.rs	1988	f857f488d2056e7316e3b89db35e8d3cb1204835a3d10adb89b3cf1561433e3a	0
hashes/src/sha1/mod.rs	2243	15abaf4b771584a185692fb8dc6dce03d9ca2ec93adf302038dc9a01951d7962	0
hashes/src/sha256/crypto/arm_sha2.rs	38500	761196ba342de96d9b1ac73968e11711aa65e9e561cc55eac139d749df2b57dc	0
hashes/src/sha256/crypto/avx2.rs	22485	0c0bc9ec1a651d3699c6af599445576fde79a72dcc7e2f59ab168bfa50b56fc4	0
hashes/src/sha256/crypto/mod.rs	42919	8cf411df2a92cb7f7f4abe7b45ba1941dbb491756f37f7ed0c695c7528f92255	0
hashes/src/sha256/crypto/sse41.rs	21527	cc91be883a03721e7a4dcabef558d0e2c06cea9244a4fe7188cfacee9e41afc4	0
hashes/src/sha256/crypto/x86_shani.rs	50170	e84adbbc18c6e86ebe24f42405367a2b134460b3d7f66bb5db3abedba5f692c5	0
hashes/src/sha256/mod.rs	13393	7d5adb9d3dbe4beecc65f5853b2557e0b6a5a881d1e865040e364091aa2834f3	0
hashes/src/sha256d/mod.rs	5511	8cb6e7fff0a47c6dab928bddfddcd06983bb1560d9f43440ba1c838740ec3ba4	0
hashes/src/sha256t/mod.rs	9208	f0cd394d6e9d5c8e30b22b574dde121690bfb31fcf4d28e72c1ba6e4541673c8	0

hashes/src/sha384/mod.rs	5768	3dc8e584db2516a8b3f7773ede3af349210d3a12f905bdb935b16101883eed73	0
hashes/src/sha3_256/mod.rs	7406	4131dcdc92f67cf5d4b1d872e58a7d878c9a6cfd0278e61ce94fda0d636c7596	0
hashes/src/sha512/crypto.rs	11306	a5b91b2dfe9ccadfdd7f08b2445222f5600d555bef9461326a2bc0e602e1d064	0
hashes/src/sha512/mod.rs	3685	9db14d804dcd9fd3597fc8b4a521bbabfcf32072157944bf54b0669357ece0cc	0
hashes/src/sha512_256/mod.rs	5859	fde2f1e7eeb3df2a7f7dd50514f0e985ca9233a3bffe4ded58ce1603b34870aa	0
hashes/src/siphash24/mod.rs	12498	3263477af8a6419895b6990cd709dd008d379d036e63eb220d27a23c4f51c87b	0
key_expression/Cargo.toml	2024	910599a1bfa3917a3d5f398e4c9e3640ceab90b1cefffe7752d449079c6140a3	0
key_expression/src/bip32.rs	92318	01e732b72bc832c6cd2ba7d7ba45432371a357e78231dbe4acb0578aaf148c13	0
key_expression/src/lib.rs	852	e8867d4c237adc41047ad01c18b3d99478d39275788f1f284930eaed4000b18d	0
primitives/Cargo.toml	2178	ff4824a9b6e2ec38e04fe1416ba8aa862f49572cc7c8793555c89e394d8eb1d2	0
primitives/src/block.rs	98351	15f208bbb3aaf3c5170559ab41c538f6573036e3c07a58878cfd0e9fe224e6d9	0
primitives/src/hash_types/block_hash.rs	3181	33350e38b3fcd4907c3453dca56397f4138952f9cbeaa1571737631af4c95700	0
primitives/src/hash_types/generic.rs	2089	f8733916256247da37a781222f0f80fbf0b86c45f0c51faf9c58766aa296e0eb	0
primitives/src/hash_types/mod.rs	13053	818f9b21f2a72247e3e33199879f6062d91dc08ae075651497c6c69164530fcb	0
primitives/src/hash_types/ntxid.rs	1217	c670e29ca3c15ea261e8246fd7656eeb76f1bd430216f16248bfed6ea91bcbcd	0
primitives/src/hash_types/script_hash.rs	6666	422524ffd76c13071f99b40627061bdd68bc72299e9e4bb50b482c2810d4109f	0
primitives/src/hash_types/transaction_merkle_node.rs	4299	59fd7e0db3b9551c3bbd5b98c7e0971e82c5d347492a7e6af8f18968aa89c328	0
primitives/src/hash_types/txid.rs	1260	5c70971e351eed37777c5e27effa6c1196c6351701314d43656f1aa630e8f391	0
primitives/src/hash_types/witness_commitment.rs	610	e7cf954fbd63817a7bb6ae06fae899510bc5603820c73ce27d4cbf3daeb20c31	0
primitives/src/hash_types/witness_merkle_node.rs	5181	3c574b3459d7918ef31732c30ed0568ed88a3066b7d56d14db10ef2283b64030	0
primitives/src/hash_types/witness_script_hash.rs	4186	5c4d78d2598391a44c2abf2fc2e33c28067de2ea94bcd5f9a975506095ac4c86	0
primitives/src/hash_types/wtxid.rs	973	517f27799abd0504c97d86cee83b758f29296d84990458eb28ae56004bfeafa2	0
primitives/src/hex_codec.rs	4135	5e49a620a0cd54f3aa3a4330695be29ba5e23411a46955c96d355f4f8c4a80e3	0

primitives/src/lib.rs	5486	5743595c9efea8947ced95d4b7c0370a9ebae1080a64440be3872a6c24f59b48	0
primitives/src/merkle_tree.rs	16002	d6449e5acf7c61fde7a8144ecb8d304076095ffdde66c1ebdc24e14a786d71be	0
primitives/src/opcodes.rs	15187	7f21910077ea25e9c7215359b6c8e040193852d2fb9c987efafd18e7880fb4af	0
primitives/src/script/borrowed.rs	13985	459a937a9ead0385129d5352459766785ff4b871087cac0b4ce1f80eab9f2888	0
primitives/src/script/builder.rs	7472	9153dfc34c2ede56539d00e6db0bb7b39a09450a07b71249b782740fe82bb729	0
primitives/src/script/error.rs	1063	4e4bf6dab86b801bac29e49fafec70306e251860cb6a65f98c4a724e84a9972c	0
primitives/src/script/mod.rs	20220	f3bde67ce4de1e2dd6b976d5941f8b4c08587634176ca8e3d60422a2cf54d1e7	0
primitives/src/script/owned.rs	18360	042ede48727e659bc4c180b41676b9521c8bd00f41c20c3ab509d11c77b4e000	0
primitives/src/script/push_bytes.rs	23070	c8c00a961c670c77bc0d43e95191eb743c786c6ed6baa16d3f855c4248ce3b60	0
primitives/src/script/tag.rs	1727	bf739230a9e9401bf9b513c2dc4baccfde09db341ab47900f3c1195b17d635fa	0
primitives/src/transaction.rs	135524	6439681b5d335decd22d56742c46855477b3965a896c205b547a1dca4afea1c1	0
primitives/src/witness.rs	74499	1e8cb120552a2679ee86feef0406003e329d67361557735917d732e111c6604d	0
primitives/src/witness_version.rs	14525	ac3e6a532a41af4f690315de94ae4b26665a863d3737587eaf29b8a8cae4c565	0
taproot_primitives/Cargo.toml	1696	53e2d6683ce90b7395196b769b66d6e16e025833472b52982f47c1e784dd9c3a	0
taproot_primitives/src/lib.rs	21276	0efe9ae04fe70035b6d6d9d56defe9de124c1e49f474c63c6d99e055341295ac	0